Acelerador Sistólico 8×8 INT8

Registradores, Sinais de Controle e Status
Integração com o SoC NEORV32 (DE2-115 / Cyclone IV E)

Vinícius Levi

12 de agosto de 2026

Sumário

1. Visão Geral
2. Mapa de Registradores
3. Sinais de Controle e Status Internos
4. Interface Wishbone
5. Máquina de Estados (FSM)
6. Processing Element (PE) e Array
7. Temporização
8. Sequência de Operação
9. Referência Consolidada

O que é o acelerador

Função

Multiplicador de matrizes **INT8** $\rightarrow$ **INT32** baseado em *array sistólico weight-stationary* 8×8, acoplado ao NEORV32 como periférico mapeado em memória.

Arquivo RTL efetivamente integrado

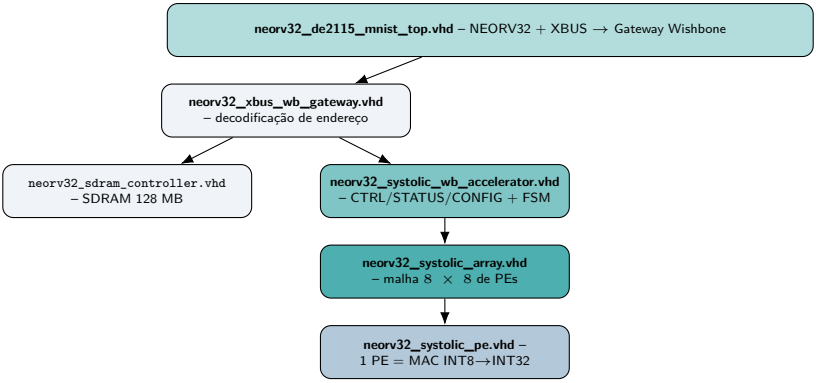
`neorv32_systolic_wb_accelerator.vhd` (interface Wishbone clássica, substitui o antigo `neorv32_cfs.vhd`)

- Opera sobre **tiles** 8×8: um tile de pesos (*stationary*) e um tile de ativações.
- Resultado: `RESULT = ACT @ WEIGHT`, INT32, 8×8.
- $\text{RESULT}[r, c] = \sum_{k=0}^{7} \text{ACT}[r, k] \cdot \text{WEIGHT}[k, c]$
- Usado para acelerar camadas de CNN/FC (inferência MNIST) – ver `sw/example/neorv32_mnist_infer`.

Parâmetros

Linhas / Colunas	8×8
Dado de entrada	INT8
Acumulador	INT32
PEs totais	64

Hierarquia de módulos



Endereçamento no SoC (barramento XBUS)

Faixa de endereço	Tamanho	Destino
0x80000000 – ...	(reservado)	DMEM interno do NEORV32
0x90000000 – 0x9000FFFF	64 KB	**Acelerador sistólico**
0xA0000000 – 0xA7FFFFFF	128 MB	SDRAM (pesos/ativações/buffers)

Gateway XBUS → Wishbone (`neorv32_xbus_wb_gateway.vhd`)

- Decodifica `xbus_adr_i` e roteia `stb`/`cyc` para o escravo correspondente.
- Endereço relativo: `accel_adr_o = xbus_adr_i - 0x90000000`.
- Acesso fora das janelas mapeadas gera `xbus_err_o = '1'`.

Mapa de memória do acelerador

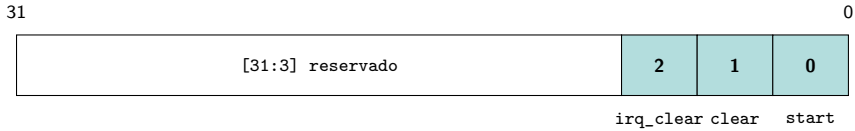
Base: `ACCEL_BASE = 0x90000000`. Endereçamento por *word* de 32 bits.

Offset	Nome	Acesso	Descrição
0x000	`CTRL`	R/W	Controle: start, clear, ack IRQ
0x004	`STATUS`	RO	busy, done, irq_pending
0x008	`CONFIG`	R/W	Reservado (uso futuro)
0x100 -- 0x13C	`WEIGHT[0..15]`	R/W	Tile de pesos 8×8 INT8 (64 B)
0x500 -- 0x53C	`ACT[0..15]`	R/W	Tile de ativações 8×8 INT8 (64 B)
0x900 -- 0x9FC	`RESULT[0..63]`	RO	Resultado 8×8 INT32 (256 B)

Nota de implementação

Índices de registrador na RTL usam `wb_adr_i(15 downto 2)` (constantes `REG_CTRL`, `REG_STATUS`, `WEIGHT_BASE_W`, `ACT_BASE_W`, `RESULT_BASE_W` em `neorv32_systolic_wb_accelerator.vhd:73--78`).

Registrador CTRL – 0x000 (R/W)

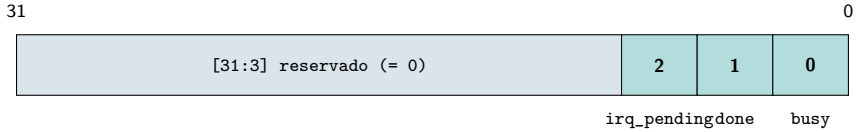


Bit	Campo	Descrição
[0] start	start	Pulso para iniciar a computação. FSM só reage se array_busy = 0 e array_clear = 0 (start_req).
[1] clear	clear	Nível: enquanto em '1', mantém o array sistólico em reset funcional (array_clear), zerando acumuladores e pesos carregados.
[2] irq_clear	ack_irq	Escrever '1' limpa a flag irq_pending e desativa irq_o.
[31:3] –	reservado	Sem efeito; leitura devolve o último valor escrito em ctrl_reg.

Observação:

ctrl_reg é um registrador simples (não auto-clear); o firmware tipicamente escreve 0x1 para start e depois pode escrever 0x0 ou deixar o bit setado – o pulso real de início (array_start) é gerado pela FSM, não pelo bit em si.

Registrador STATUS – 0x004 (somente leitura)



Bit	Campo	Descrição
[0] busy	busy	Espelha array_busy: computação em andamento (carregando, somando, ou aguardando pipeline).
[1] done	done	Espelha array_done: pulso/nível de conclusão do array para a linha de ativação atual.
[2] irq_pending	irq_pending	'1' após borda de subida de array_done; permanece até CTRL[2]=1.
[31:3] –	reservado	Fixo em '0'.

Uso típico (polling, sem IRQ conectada no top atual)

```
while (ACCEL_STATUS & 0x01);  // aguarda busy==0
while (!(ACCEL_STATUS & 0x02));  // aguarda done==1
```

Registrador `CONFIG` – 0x008 (R/W)

- Implementado como registrador **livre** de 32 bits, sem lógica associada no datapath atual.
- Leitura devolve exatamente o último valor escrito (*loopback*).
- Reservado para extensões futuras: seleção de modo, tamanho de tile variável, escala/quantização programável, etc.
- Contraste: em `neorv32_systolic_top.vhd` (variante alternativa, não integrada ao SoC atual) o `CONFIG` já reserva os campos `[31:16]` rows / `[15:0]` cols para operação em tiles.

Janelas de dados: WEIGHT, ACT, RESULT

WEIGHT – 0x100 (R/W)

- 16 *words* de 32 bits (4 bytes INT8 cada) = 64 B = tile 8×8.
- `wb_sel_i` seleciona quais dos 4 bytes da word são escritos (byte-enable).
- Layout *row-major*: `weight_mem[row*8+col]`.

ACT – 0x500 (R/W)

- Mesmo formato do WEIGHT: 16 words, 64 B, row-major.
- `act_mem[row*8+col]`.

RESULT – 0x900 (somente leitura)

- 64 *words* de 32 bits (INT32) = 256 B.
- `result_mem[row*8+col] = RESULT[row][col]`.
- Escritas nesta janela são aceitas no barramento (geram ack) mas **não têm efeito**.

Byte-enable na escrita

```
for i in 0 to 3 loop
  if (wb_sel_i(i) = '1') then
    weight_mem((addr_idx-WEIGHT_BASE_W)*4+i)
      <= wb_dat_i((i+1)*8-1 downto i*8);
  end if;
end loop;
```

Sinais internos: acelerador ↔ array sistólico

Sinal	Direção	Origem	Descrição
`array_start`	accel → array	FSM (estado `S_LOAD_ACTS`)	Pulso de 1 ciclo; dispara o carregamento de pesos (se ainda não carregados) e o início do cômputo de uma linha.
`array_clear`	accel → array	`ctrl_reg(1)` (combinacional)	Nível; zera acumuladores, registradores de peso e de *skew* enquanto ativo.
`array_busy`	array → accel	`busy_reg`	`'1'` do início de `start_i` até completar `ARRAY_ROWS+ARRAY_COLS-1` ciclos de pipeline.
`array_done`	array → accel	`done_reg`	Pulsa `'1'` por 1 ciclo quando `cycle_cnt` atinge o valor de latência total.

Barramento	Largura	Sentido	Conteúdo
`act_cols_to_array`	$8 \times 8 = 64$ bits	accel → array	1 linha de ativações (8×INT8)
`weight_to_array`	$8 \times 8 \times 8 = 512$ bits	accel → array	tile de pesos completo (64×INT8)
`result_from_array`	$8 \times 32 = 256$ bits	array → accel	1 linha de resultado (8×INT32)

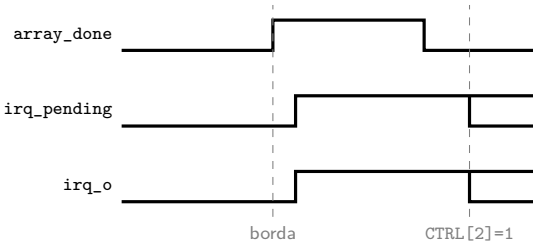
Detecção de borda e geração de IRQ

Sinais

- `prev_done`: registra `array_done` do ciclo anterior.
- `irq_pending`: seta em borda de subida (`array_done='1' and prev_done='0'`).
- `irq_o`: replica `irq_pending` (nível, não pulso).

Estado atual da integração

Em `neorv32_de2115_mnist_top.vhd`, `accel_irq` **não** está conectado a um controlador de interrupção (PLIC/FIRQ) – apenas ao LED `LEDR[1]`. O firmware da Fase 3/4 usa *polling* de `STATUS`; a IRQ fica disponível para trabalho futuro.



Portas do barramento Wishbone (escravo)

Sinal	Largura	Dir.	Função
`wb_adr_i`	32	In	Endereço (word-aligned; índice = `adr(15:2)`)
`wb_dat_i`	32	In	Dados de escrita (CPU → acelerador)
`wb_dat_o`	32	Out	Dados de leitura (acelerador → CPU)
`wb_we_i`	1	In	Write Enable (`1`=escrita, `0`=leitura)
`wb_sel_i`	4	In	Byte-select (1 bit por byte da word)
`wb_stb_i`	1	In	Strobe: ciclo válido
`wb_cyc_i`	1	In	Cycle: barramento em uso
`wb_ack_o`	1	Out	Acknowledge: transação concluída (pulso 1 ciclo)
`wb_err_o`	1	Out	Error (não utilizado pelo acelerador; sempre `'0'`)
`irq_o`	1	Out	Interrupção para o processador (não conectada no top atual)

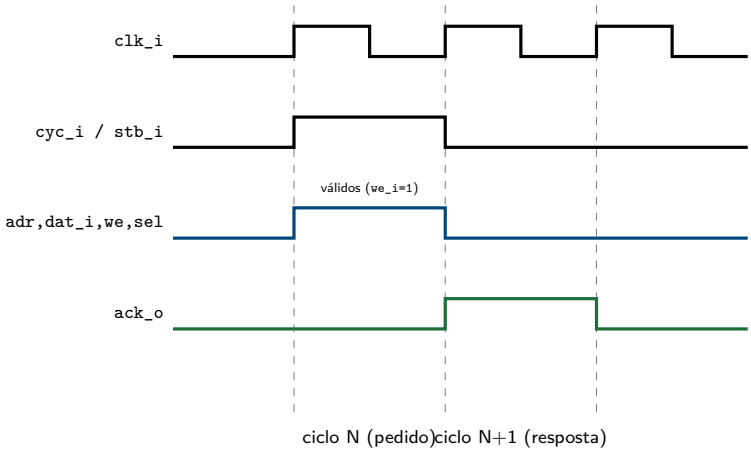
Protocolo

Modo *registered feedback*: mestre mantém `cyc=stb=1` por 1 ciclo; escravo responde `ack=1` no ciclo seguinte, junto com o dado (`wb_dat_o`) em caso de leitura.

Protocolo Wishbone clássico – fundamentos

- **Síncrono**: todos os sinais são amostrados/gerados em borda de subida de `clk_i`; não há lógica assíncrona no barramento.
- **Single-beat, não pipelined**: cada transação move exatamente 1 word de 32 bits; o próximo ciclo só começa depois que o atual termina (`ack` ou `err`). Não há sobreposição de transações.
- **Sem burst**: o acelerador não decodifica `cti` (cycle type) – cada pulso de `cyc&stb` é tratado como uma transação independente.
- **Registered feedback**: a resposta (`ack`/`dat_o`) é gerada por um registrador no escravo, nunca combinacionalmente a partir do endereço – daí o nome: o *feedback* do escravo é sempre registrado, o que impõe no mínimo 1 ciclo de espera (*wait state*) por transação.
- **Neste acelerador, a latência é fixa**: `wb_ack_o` é sempre `'1'` no ciclo seguinte a `cyc&stb` – o escravo nunca insere *wait states* extras (todo acesso, incluindo escrita em memória, completa em exatamente 1 ciclo).
- O protocolo, porém, **suporta** escravos de latência variável: o mestre XBUS do NEORV32 mantém `cyc` ativo até receber `ack` ou `err`, não importa quantos ciclos isso leve (ver próximos slides).

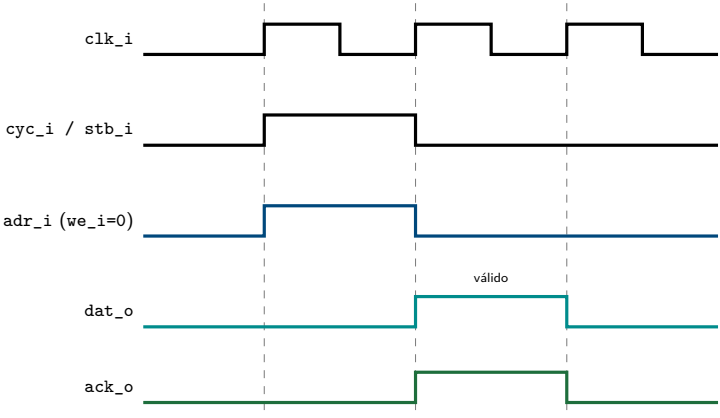
Temporização: ciclo de escrita



Ciclo de escrita – passo a passo

- Mestre coloca `adr_i`, `dat_i`, `we_i=1` e `sel_i` estáveis **junto** com `cyc_i=stb_i=1`, no ciclo N.
- Na borda de subida ao final do ciclo N, o escravo amostra tudo isso e grava internamente (`weight_mem`/`act_mem`/`ctrl_reg`...).
- `ack_o` sobe no ciclo N+1, confirmando a escrita; o mestre então derruba `cyc_i`/`stb_i`.
- Sem essa confirmação, o mestre (via `pending` no XBUS) manteria o ciclo aberto indefinidamente – `ack` é o que libera o barramento para a próxima transação.

Temporização: ciclo de leitura



Ciclo de leitura – passo a passo

- Mestre coloca apenas `adr_i` e `we_i=0` (dado de escrita irrelevante) junto com `cyc_i=stb_i=1` no ciclo N.
- O escravo decodifica `addr_idx` e seleciona a fonte (`ctrl_reg`, `status_reg`, `result_mem`, …).
- `dat_o` e `ack_o` tornam-se válidos **juntos** no ciclo N+1 – ambos são atribuídos no mesmo processo registrado (`wb_slave`), então chegam sincronizados ao mestre.
- Se o mestre precisar encadear outra leitura logo em seguida, o novo `adr_i` do ciclo N+1 **já pode divergir** do endereço que gerou este `dat_o` – por isso o gateway precisa da seleção registrada (`sel_*_r`) ao rotear a resposta (ver próximo slide).

Sincronização mestre → gateway → escravo

Mestre XBUS (neorv32_xbus.vhd)

- Registrador `pending` sobe junto com o 1º `stb` e mantém `cyc_o = stb_o or pending` ativo até `ack_i` ou `err_i`.
- Isso permite escravos de **latência variável** sem que o mestre precise saber quantos ciclos vai levar.

Gateway (neorv32_xbus_wb_gateway.vhd)

- Decodifica `sel_sdram`/`sel_accel` **combinacionalmente** a partir do endereço atual.
- Mas **registra** essa escolha em `sel_*_r` no ciclo do pedido – usada para rotear a resposta 1 ciclo depois.

```
track_sel: process(clk_i)
begin
 if rising_edge(clk_i) then
  if (xbus_cyc_i='1' and
      xbus_stb_i='1') then
   sel_sdram_r <= sel_sdram;
   sel_accel_r <= sel_accel;
  end if;
 end if;
end process;

xbus_dat_o <=
  sdram_dat_i when sel_sdram_r='1'
  else accel_dat_i when sel_accel_r='1'
  else (others=>'0');
```

Por que registrar a seleção?

O problema: o endereço pode mudar antes da resposta chegar

A resposta do escravo (`dat_i`/`ack_i`) só chega **1 ciclo depois** do pedido. Se o endereço mudar nesse meio-tempo (próxima transação já em andamento), decodificar o endereço *atual* rotearia a resposta **antiga** para o destino **errado**.

A solução: `sel_sdram_r` / `sel_accel_r`

O gateway "lembra", em um registrador capturado no próprio ciclo do pedido, qual escravo foi endereçado. Quando a resposta chegar (1 ciclo depois), o mux de saída (`xbus_dat_o`, `xbus_ack_o`, `xbus_err_o`) usa essa cópia registrada – não o endereço atual, que já pode pertencer à transação seguinte.

Esse é o mesmo princípio por trás de qualquer barramento compartilhado com múltiplos escravos e resposta registrada: o roteamento da resposta precisa de memória de qual pedido está em voo, não apenas do estado atual do endereço.

Assimetria de latência: ack vs err

```
xbus_ack_o <= (sdram_ack_i and sel_sdram_r) or (accel_ack_i and sel_accel_r);

xbus_err_o <= (sdram_err_i and sel_sdram_r) or (accel_err_i and sel_accel_r) or
              (xbus_cyc_i and xbus_stb_i and not sel_sdram and not sel_accel);
```

ack – endereço mapeado

Depende de `sel_sdram_r`/`sel_accel_r` (**registrados**): chega no mínimo 1 ciclo depois do pedido, no ritmo do escravo alvo.

err – endereço fora de faixa

O último termo usa `sel_sdram`/`sel_accel` **combinacionais** (não registrados): um acesso a região não mapeada gera `err` no **mesmo ciclo** do pedido – 0 wait states, mais rápido que um `ack` válido.

Consequência: o barramento reage mais rápido a um endereço inválido do que a um válido – a CPU (via `bus_rsp.err` em `neorv32_xbus.vhd`) recebe o erro de barramento sem esperar o timeout do XBUS (`XBUS_TIMEOUT`).

Byte-select e domínio de clock

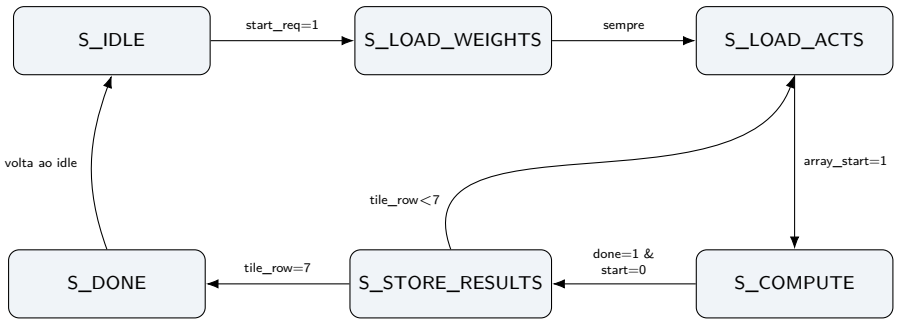
wb_sel_i (byte-enable) – assimetria escrita/leitura

- Em WEIGHT/ACT: cada um dos 4 bits de sel_i habilita a escrita de 1 byte específico (mascaramento real, via for).
- Em CTRL/CONFIG: a word inteira de 32 bits é escrita (ctrl_reg <= wb_dat_i) – sel_i é **ignorado**.
- Em **qualquer leitura**: a word de 32 bits completa é sempre devolvida; sel_i não afeta wb_dat_o. Cabe ao LSU do NEORV32 aplicar a máscara de byte/half-word do lado do mestre.

Um único domínio de clock

- CPU, XBUS, gateway, acelerador, array sistólico e PEs operam todos síncronos ao mesmo clk_i (50 MHz, direto do oscilador da placa) – **não há travessia de domínio de clock (CDC)** no caminho do barramento Wishbone deste projeto.
- O único sincronizador do top-level é para o **reset externo** (botão físico KEY(0)): 2 flip-flops em cascata (sync_rst em neorv32_de2115_mnist_top.vhd) – não relacionado ao protocolo Wishbone, apenas garante que rstn_i chegue livre de metaestabilidade a todos os módulos.

FSM de computação – diagrama de estados



Cada pulso de `array_start` computa 1 linha; a FSM percorre `S_LOAD_ACTS` → `S_COMPUTE` → `S_STORE_RESULTS` 8 vezes (1 por `tile_row`) antes de alcançar `S_DONE`.

FSM de computação – ação por estado

Estado	Ação
S_IDLE	Aguarda start_req = CTRL[0] & !busy & !clear.
S_LOAD_WEIGHTS	Copia weight_mem → weight_to_array (tile inteiro, 1× por operação).
S_LOAD_ACTS	Copia a linha tile_row de act_mem → act_cols_to_array; pulsa array_start.
S_COMPUTE	Aguarda array_done='1' and array_start='0'; ao concluir, grava a linha em result_mem.
S_STORE_RESULTS	Decide se ainda há linhas (tile_row < 7) → volta a S_LOAD_ACTS, senão → S_DONE.
S_DONE	Um ciclo; retorna a S_IDLE.

Guarda de corrida em S_COMPUTE

Bug real corrigido (encontrado via GHDL, neorv32_systolic_matmul_tb)

A condição de transição usa `array_done='1' and array_start='0'`, não apenas `array_done='1'`.

- No ciclo em que `array_start` é pulsado (entrando em `S_COMPUTE` vindo de `S_LOAD_ACTS`), o array ainda não processou o pulso – seu `done_reg` só zera no ciclo seguinte.
- Sem a guarda, um `done_reg` obsoleto (ainda `'1'` da linha anterior) faz a FSM concluir precocemente que a linha atual já terminou – captura o resultado após 1 ciclo em vez dos 15 necessários.
- Só a linha 0 batia por acaso (pipe zerado pelo `reset` global); linhas seguintes divergiam sistematicamente.

Sinais do PE – neorv32_systolic_pe.vhd

Porta	Larg.	Descrição
en_i	1	Habilita acumulação (= busy_reg do array)
clr_i	1	Zera acc_reg, a_reg, w_reg
a_in_i	8	Ativação de entrada (INT8)
w_in_i	8	Peso de entrada (INT8)
p_in_i	32	Soma parcial de entrada (do PE acima)
a_out_o	8	Ativação propagada (para a direita)
w_out_o	8	Peso propagado
p_out_o	32	Soma parcial de saída (para o PE abaixo)

Datapath (1 ciclo)

$$\texttt{mult_res} = \texttt{a_in_i} \times \texttt{w_in_i}$$

$$\texttt{acc_reg} \leftarrow \texttt{p_in_i} + \texttt{mult_res}$$

MAC INT8×INT8→INT32, registrado, sem saturação (resize para 32 bits assume que não há overflow para tiles 8×8).

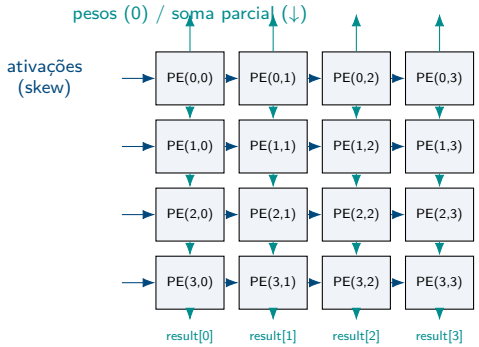
Sinais de controle internos do array – neorv32_systolic_array.vhd

Sinal	Tipo	Papel
busy_reg	FF	Espelhado em busy_o; ativo do start_i até completar a latência de pipeline.
done_reg	FF	Espelhado em done_o; pulsa quando cycle_cnt atinge ARRAY_ROWS+ARRAY_COLS-1.
cycle_cnt	contador	Conta ciclos desde start_i; range 0..ARRAY_ROWS+ARRAY_COLS+1.
weight_loaded	FF	Gate de w_in_pe: PEs só recebem pesos reais após o 1º start_i (evita lixo de reset).
pe_weight_reg	array 8×8 de FF	Cópia *stationary* do tile de pesos, carregada uma vez por start_i com busy_reg=0.
act_skew_regs	array 8×8 de FF	Registradores de deslocamento (*skew*): cada linha desliza sua ativação 1 coluna/ciclo.

Reset do *skew* por-start (bug corrigido)

act_skew_regs zera não só em clear_i, mas também a cada novo start_i (busy_reg=0). Sem isso, valores "em trânsito"de uma linha anterior contaminavam a soma parcial das primeiras colunas da linha seguinte.

Fluxo de dados no array sistólico



- **Ativações**: fluem horizontalmente (esquerda→direita), *skewed* por linha – 1 elemento por linha entra na coluna 0 a cada ciclo.
- **Somas parciais**: fluem verticalmente (topo→base), acumuladas por coluna.
- **Pesos**: tile completo pré-carregado nos PEs (*weight-stationary*), 1 valor independente por PE (row, col) – não broadcast.

Latência e throughput

Latência de pipeline por linha

$$\text{ciclos} = \text{ARRAY_ROWS} + \text{ARRAY_COLS} - 1 = 8 + 8 - 1 = 15 \text{ ciclos}$$

- Cada pulso de `start_i` computa **1 linha** de ativações contra o tile de pesos (já carregado), levando 15 ciclos até `done_o='1'`.
- Uma operação completa de tile $8 \times 8 @ 8 \times 8$ dispara `array_start` **8 vezes** (1 por `tile_row`), cada uma precedida por `S_LOAD_ACTS` (1 ciclo).
- Latência total aproximada: $8 \times (15 + 1) \approx 128$ ciclos de `clk_i` por tile (mais overhead de acesso ao barramento Wishbone para carregar WEIGHT/ACT e ler RESULT).
- Pesos são carregados apenas uma vez (`S_LOAD_WEIGHTS` só ocorre na transição `S_IDLE`→`S_LOAD_WEIGHTS`).

Sequência completa de uso (firmware)

1. **Reset/clear opcional**: `CTRL = 0x2`; aguardar `STATUS.busy=0`.
2. **Carregar pesos**: 16 escritas em `WEIGHT[0..15]` (64 B, tile 8×8 INT8).
3. **Carregar ativações**: 16 escritas em `ACT[0..15]` (64 B).
4. **Disparar**: `CTRL = 0x1` (bit `start`).
5. **Aguardar conclusão**: *polling* em `STATUS.done` (bit 1) ou `STATUS.busy` (bit 0) voltando a 0.
6. **Ler resultado**: 64 leituras em `RESULT[0..63]` (INT32, row-major).
7. *(Opcional)* **Limpar IRQ**: `CTRL = 0x4` se `irq_pending` foi usado.

```
ACCEL_CTRL = 0x00000002;          // clear
while (ACCEL_STATUS & 0x01);      // aguarda busy==0
for (i=0;i<16;i++) ACCEL_WEIGHT(i) = w[i];
for (i=0;i<16;i++) ACCEL_ACT(i)    = a[i];
ACCEL_CTRL = 0x00000001;          // start
while (!(ACCEL_STATUS & 0x02));   // aguarda done==1
for (i=0;i<64;i++) result[i] = ACCEL_RESULT(i);
```

Driver de software – `accel_driver.c/.h`

- `ACCEL_BASE = 0x90000000u`
- API de alto nível: `accel_matmul_i8(A, M, K, W, N, zp_a, C)`
- Multiplica $A_{M\times K} \times W_{K\times N}$ (INT8) $\rightarrow C_{M\times N}$ (INT32, RAW).
- Aplica *zero-point* de entrada (`zp_a`) – quantização assimétrica estilo TFLite; peso assumido com zero-point 0.
- Faz *tiling* automático: M, K, N não precisam ser múltiplos de 8 – bordas preenchidas com zero.
- Bias e requantização ficam a cargo da camada acima (`inference.c`).

Tabela-resumo: todos os registradores

Offset	Reg.	Bits	Significado
0x000	CTRL	[0]	`start` – pulso de início
		[1]	`clear` – reset funcional do array (nível)
		[2]	`irq_clear` – confirma/limpa IRQ
		[31:3]	reservado
0x004	STATUS	[0]	`busy` – computação em andamento
		[1]	`done` – linha concluída
		[2]	`irq_pending` – IRQ pendente
		[31:3]	reservado (sempre 0)
0x008	CONFIG	[31:0]	livre / reservado (uso futuro)
0x100–-0x13C	WEIGHT[0..15]	[31:0] × 16	tile de pesos 8×8 INT8
0x500–-0x53C	ACT[0..15]	[31:0] × 16	tile de ativações 8×8 INT8
0x900–-0x9FC	RESULT[0..63]	[31:0] × 64	resultado 8×8 INT32

Sinal interno	Módulo	Papel
`array_start / array_clear`	wb_accelerator → array	controle do array
`array_busy / array_done`	array → wb_accelerator	status do array
`irq_pending / irq_o`	wb_accelerator	IRQ nível, latch por borda de `done`
`fsm_state / tile_row`	wb_accelerator	FSM de orquestração (6 estados)
`busy_reg / done_reg / cycle_cnt`	array	controle de pipeline (15 ciclos)
`weight_loaded / pe_weight_reg`	array	tile de pesos *stationary*
`act_skew_regs`	array	deslocamento (skew) das ativações
`acc_reg / mult_res`	PE	MAC INT8→INT32

Referências e arquivos-fonte

- `rtl/core/neorv32_systolic_wb_accelerator.vhd` – registradores, FSM, interface Wishbone **(módulo integrado ao SoC atual)**
- `rtl/core/neorv32_systolic_array.vhd` – malha 8×8, skew, pipeline
- `rtl/core/neorv32_systolic_pe.vhd` – elemento de processamento (MAC)
- `rtl/core/neorv32_xbus_wb_gateway.vhd` – decodificação de endereço XBUS
- `rtl/core/neorv32_de2115_mnist_top.vhd` – topo do SoC (DE2-115)
- `sw/example/neorv32_mnist_infer/accel_driver.[ch]` – driver de software
- `sw/example/test_systolic_accel/main.c` – teste unitário de registradores
- `sim/neorv32_systolic_matmul_tb.vhd` – testbench GHDL (encontrou os 2 bugs de corrida citados)
- `docs/Systolic Array Guide.md` – guia estendido em Markdown

Obrigado!

Dúvidas?